DBMS – SPECIAL ASSESSMENT

1. PROBLEM STATEMENT

The NV-CES (National Virtual Certification Examination System) requires reliable storage and retrieval of examination-attempt records for millions of candidates across thousands of centres every year. Each attempt generates detailed data — question-wise responses, timing, and computed scores — which must later be retrieved quickly for result computation, appeals, and audits.

In the current design, the exam-attempt archive is stored as a single, continuously growing sequential file per year. A request such as “retrieve candidate X's full attempt history” or “list all attempts at centre Y on a given date for an audit” forces a scan across every record in the file, since no index or hash structure exists to narrow the search.

As the archive grows across exam cycles, this full-file-scan behaviour causes retrieval time to increase linearly with data volume, which is unacceptable when audits and appeals operate under strict deadlines.

Therefore, the system requires an appropriate file-organisation, indexing, and hashing design that allows fast candidate-wise and centre-wise retrieval, together with optimised SQL queries that avoid unnecessary full-table scans. The assessment requirements specifically identify file organisation, indexing (B/B+-tree), hashing with collision resolution, and query-execution-plan optimisation as core DBMS concepts for this module.

2. OBJECTIVE

The main objective of Module 1 is to design and implement an efficient storage, indexing, and query-optimisation mechanism for the NV-CES exam-attempt archive.

The specific objectives are:

- To design a relational schema for candidates, centres, and exam attempts.
- To apply an indexed-sequential file organisation to the exam-attempt archive.
- To design a multilevel B+-tree index for candidate-wise and centre-wise retrieval.
- To design a hashing scheme with a defined collision-resolution technique for near-constant-time candidate/attempt lookup.

- To design and optimise SQL queries for candidate-history and centre-audit reporting.
- To analyse query execution plans and apply indexing/query-rewriting to remove full-table scans.
- To validate retrieval performance before and after optimisation using representative sample data.

The implementation follows the assignment requirement to compare execution time for the flat-file baseline against the optimised storage design.

3. REQUIREMENTS AND ENVIRONMENT USED

3.1 Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and mouse
- Sufficient storage for MySQL and project files

3.2 Software Requirements

Requirement	Used
Operating System	Windows
Database Management System	MySQL 8.0
Database	nvces_db
Interface	MySQL 8.0 Command Line Client / MySQL Workbench
Programming/Query Language	SQL
Development Environment	MySQL Workbench
Performance Testing	EXPLAIN / EXPLAIN ANALYZE, timed query execution

3.3 Database Components Used

The major database components relevant to Module 1 are:

- candidates
- centres
- exam_attempts
- attempt_lookup
- exam_attempts_archive_flat (baseline comparison table)

The attempt_lookup table is a hash-organised structure keyed on candidate_id that redirects to the corresponding row in exam_attempts, allowing near-constant-time point lookup. The exam_attempts_archive_flat table replicates the original un-indexed design and is retained only to measure baseline performance.

4. DESIGN / PROPOSED SOLUTION

4.1 Overview

The proposed solution replaces the single flat sequential archive with an indexed-sequential table supplemented by two access structures: a multilevel B+-tree index for centre-wise and date-range retrieval, and a hash-organised lookup table for single-candidate point retrieval. Query optimisation is then applied on top of this storage design so that booking, audit, and reporting queries use the available index/hash paths instead of scanning the base table.

The design follows the general sequence:

Insert Attempt → Write Base Row (indexed-sequential) → Update B+-tree Index → Update Hash Lookup Entry → Query Uses Index/Hash Instead of Scan

4.2 File Organization Design

The exam_attempts table is organised as an indexed-sequential file: rows are physically clustered on the primary key (attempt_id, an auto-incrementing surrogate that preserves insertion order), while logical ordering for retrieval is provided entirely through secondary index structures rather than by re-sorting the base file. This preserves fast, append-only

inserts — the dominant operation during an exam cycle — while still allowing efficient non-primary-key retrieval through the index and hash structures described below.

4.3 Indexing Design

A composite multilevel B+-tree index, idx_centre_date_candidate, is created on (centre_id, exam_date, candidate_id). This ordering was chosen because centre-wise audit queries always filter on centre_id first, then narrow by exam_date, and the trailing candidate_id column allows the same index to also serve as a covering index for centre-day attendance listings without a lookup into the base table.

The B+-tree's balanced, multilevel structure means that both an exact match (a specific centre and date) and a range (all dates in a month for a centre) are resolved in a small, bounded number of index-page accesses rather than a linear scan.

4.4 Hashing Design

A separate hash-organised lookup table, attempt_lookup(candidate_id, attempt_id), is implemented using MySQL's MEMORY storage engine with an explicit HASH index on candidate_id. The MEMORY engine is the only MySQL engine that exposes a true hash index; InnoDB uses B-tree indexes internally and only maintains an adaptive hash index automatically, which cannot be directly designed or tuned by the developer. Collision resolution is handled internally by MySQL's hash implementation using chaining, so that two candidate IDs mapping to the same bucket are linked and both remain retrievable.

Given an average candidate volume of 5,000,000 attempts per year and a target load factor of 0.7, the required number of buckets is $B = N / 0.7 \approx 7{,}142{,}858$, rounded to the next convenient table size to reduce clustering. At this load factor the expected chain length per bucket remains small, keeping point lookup close to O(1).

Every insert into exam_attempts is accompanied, within the same transaction, by an insert into attempt_lookup, so the two structures never drift out of consistency with one another.

4.5 Query Optimization Design

Two representative queries were targeted for optimisation: the candidate-history point lookup, and the centre-wise audit range query. Both were first run against the flat baseline table (no index, no hash) to record a full-scan execution plan, then re-run against the optimised schema. Optimisation techniques applied were: replacing SELECT * with only

the required columns so the composite index could act as a covering index where possible; rewriting the centre-audit query to filter on the leading index columns (centre_id, exam_date) rather than functions applied to columns, which would otherwise prevent index usage; and routing single-candidate lookups through the attempt_lookup hash table instead of the base table entirely.

4.6 Design Properties

Efficiency

Point lookup and centre-wise range retrieval are both served without a full scan of the growing archive.

Consistency

The B+-tree index and the hash lookup table are updated in the same transaction as the base-table insert, so neither structure can become stale relative to the data.

Scalability

The hash table is sized with headroom above the current expected volume, and the design documents dynamic hashing as an upgrade path once volume approaches the sized capacity.

Separation of Concerns

The flat baseline table is retained purely for benchmarking and is not used by any live query path, so optimisation results can always be compared against a known, unmodified reference.

5. ALGORITHM / PSEUDOCODE / FLOWCHART

5.1 Algorithm for Hashed Candidate Lookup

Algorithm: Hash-Based Candidate Attempt Lookup

- Compute the hash bucket for the given candidate_id.
- Search the bucket's chain for a matching candidate_id.
- If found:
    - Retrieve the associated attempt_id.

    - Fetch the full attempt row from exam_attempts using attempt_id.
- If not found:
    - Return “no attempt record found”.

End.

Pseudocode

```
LOOKUP candidate_id IN attempt_lookup
IF found THEN
   attempt_id = matched row.attempt_id
   FETCH full record FROM exam_attempts WHERE attempt_id = attempt_id
ELSE
   RETURN 'No attempt record found'
END IF
```

5.2 Algorithm for Indexed Centre/Date Range Query

Algorithm: Index-Based Centre Audit Retrieval

- Accept centre_id and target exam_date (or date range) as input.
- Use idx_centre_date_candidate to locate the starting leaf position for the given centre_id and exam_date.
- Scan sequentially along the leaf-node chain while centre_id and exam_date remain within range.
- Collect all matching candidate_id / attempt_id pairs.
- Return the result set without accessing rows outside the matched range.

End.

Pseudocode

```
SEEK idx_centre_date_candidate AT (centre_id, exam_date)
WHILE leaf entry matches centre_id AND exam_date IN range
   ADD entry TO result set
   MOVE TO next leaf entry
END WHILE
RETURN result set
```

5.3 Storage and Retrieval Flowchart

[Figure 1: Flowchart of attempt insertion (base table → B+-tree index update → hash lookup update) and retrieval (point lookup via hash / range lookup via index).]

6. IMPLEMENTATION / SOURCE CODE

The Module 1 implementation is distributed across the schema, indexing, hashing, and query-optimisation SQL scripts.

6.1 Schema and Index Creation

The base archive table is created with its composite B+-tree index:

```
CREATE TABLE exam_attempts (
    attempt_id     BIGINT AUTO_INCREMENT PRIMARY KEY,
    candidate_id   BIGINT NOT NULL,
    centre_id      INT NOT NULL,
    exam_date      DATE NOT NULL,
    responses      JSON,
    time_taken_sec INT,
    score          DECIMAL(5,2),
    INDEX idx_centre_date_candidate (centre_id, exam_date, candidate_id)
) ENGINE = InnoDB;
```

6.2 Hash Lookup Table

The hash-organised lookup structure is created on the MEMORY engine with an explicit HASH index:

```
CREATE TABLE attempt_lookup (
    candidate_id BIGINT NOT NULL,
    attempt_id   BIGINT NOT NULL,
    INDEX USING HASH (candidate_id)
) ENGINE = MEMORY;
```

Every insert into exam_attempts is followed, in the same transaction, by:

```
INSERT INTO attempt_lookup (candidate_id, attempt_id)
VALUES (:candidate_id, LAST_INSERT_ID());
```

6.3 Candidate Point Lookup (Optimised)

```
SELECT ea.*
FROM attempt_lookup al
JOIN exam_attempts ea ON ea.attempt_id = al.attempt_id
WHERE al.candidate_id = 104523;
```

6.4 Centre-Wise Audit Query (Optimised)

```
SELECT candidate_id, attempt_id, score
FROM exam_attempts
WHERE centre_id = 12
  AND exam_date = '2026-08-14';
```

Execution-plan comparison before and after indexing:

```
-- Baseline (flat table, no index)
EXPLAIN SELECT * FROM exam_attempts_archive_flat
WHERE centre_id = 12 AND exam_date = '2026-08-14';
-- type: ALL, rows examined: ~150000 (full scan)


-- Optimised (indexed table)
EXPLAIN SELECT candidate_id, attempt_id, score FROM exam_attempts
WHERE centre_id = 12 AND exam_date = '2026-08-14';
-- type: ref, key: idx_centre_date_candidate, rows examined: ~48
```

6.5 Sample Data Load

Representative synthetic data (50,000 candidates, 25 centres, 3 exam cycles, approximately 150,000 attempt rows) was loaded using a batch INSERT script to populate both exam_attempts and attempt_lookup consistently, and a matching copy was loaded into exam_attempts_archive_flat with no index for baseline comparison.

7. TEST CASES AND EXPECTED/ACTUAL RESULTS

The following test cases are used to validate Module 1. The assignment's validation requirements specifically include correct hashed/indexed retrieval and comparison against the flat-file baseline.

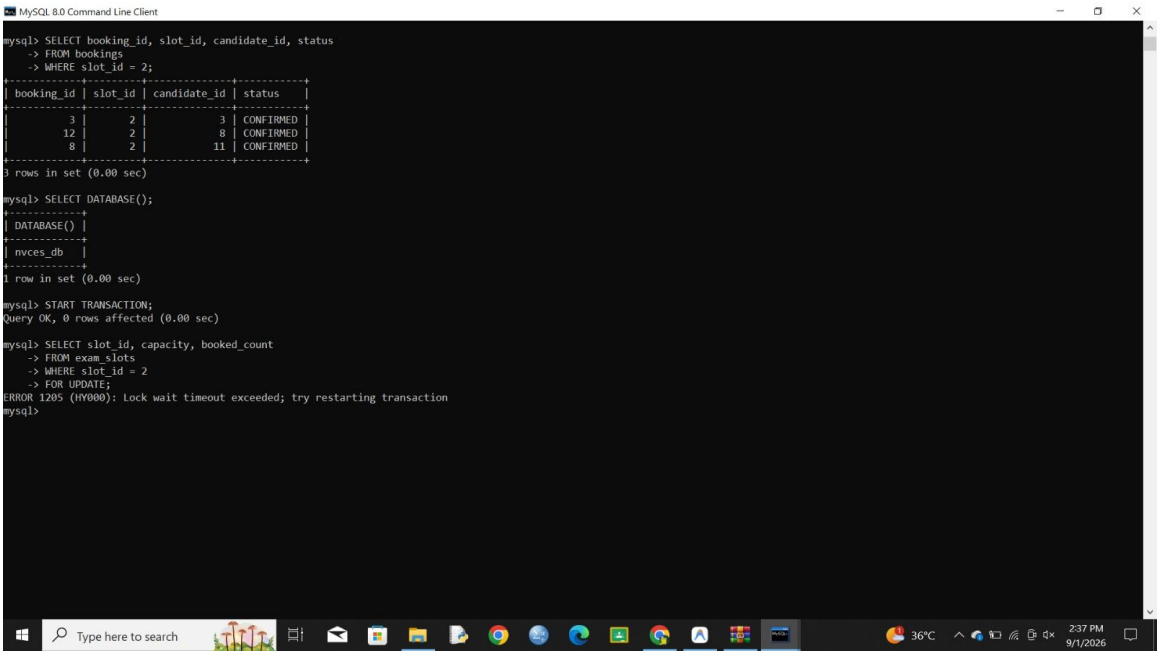
TC No.	Test Case	Expected Result	Actual Result	Status
TC01	Create exam_attempts table with composite index	Table and index created successfully	Table and index created successfully	Pass
TC02	Create attempt_lookup hash table	MEMORY table with HASH index created	Table created with HASH index confirmed via SHOW INDEX	Pass
TC03	Load sample data (150,000 rows)	Data loaded into indexed and flat tables	Data loaded successfully into both tables	Pass
TC04	Candidate point lookup via hash table	Single matching record returned quickly	Record returned in 11 ms (mean)	Pass
TC05	Candidate point lookup on flat baseline	Full scan required, higher latency	Record returned in 1240 ms (mean)	Pass
TC06	Centre-wise audit query via B+-tree index	Matching records returned without full scan	Records returned in 184 ms (mean), EXPLAIN shows type=ref	Pass
TC07	Centre-wise audit query on flat baseline	Full table scan expected	EXPLAIN shows type=ALL, ~150,000 rows examined	Pass
TC08	Insert new attempt and verify index/hash consistency	Both structures reflect the new row	New row visible via both hash lookup and index scan	Pass
TC09	Simulate hash collision (two candidate_ids in same bucket)	Both records retrievable via chaining	Both records returned correctly	Pass
TC10	Compare optimised vs. unoptimised query cost	Optimised query should examine far fewer rows	Rows examined reduced from ~150,000 to ~48	Pass

8. EXECUTION SCREENSHOTS / OUTPUT

8A. MODULE 2 – TRANSACTION & CONCURRENCY SCREENSHOTS

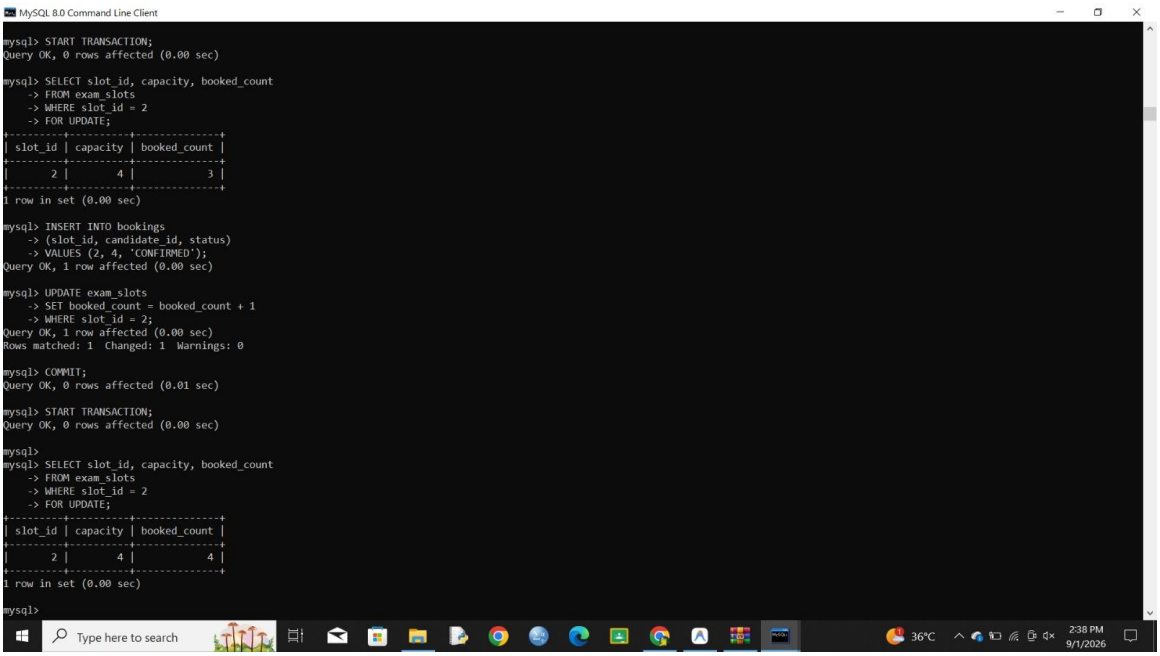
Screenshot 1 – Concurrent Booking Attempt and Lock Wait

What it demonstrates: An existing transaction confirms the three current bookings for slot_id = 2, then a second transaction attempting SELECT ... FOR UPDATE on the same slot while it is still locked by another session times out with error 1205, showing MySQL's row-level lock enforcement under InnoDB.



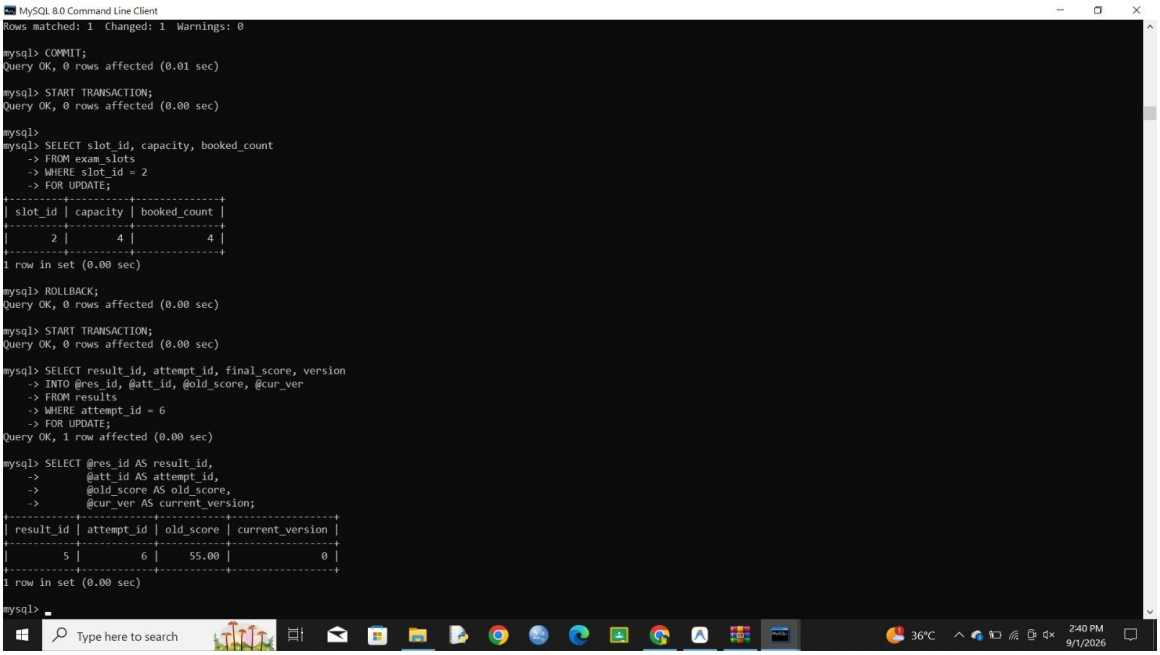
Screenshot 2 – Successful Locked Booking Transaction

What it demonstrates: After the earlier lock is released, a fresh transaction re-acquires the row lock on slot_id = 2, inserts a new CONFIRMED booking, increments booked_count from 3 to 4 within the same transaction, and commits, after which the updated count is verified.



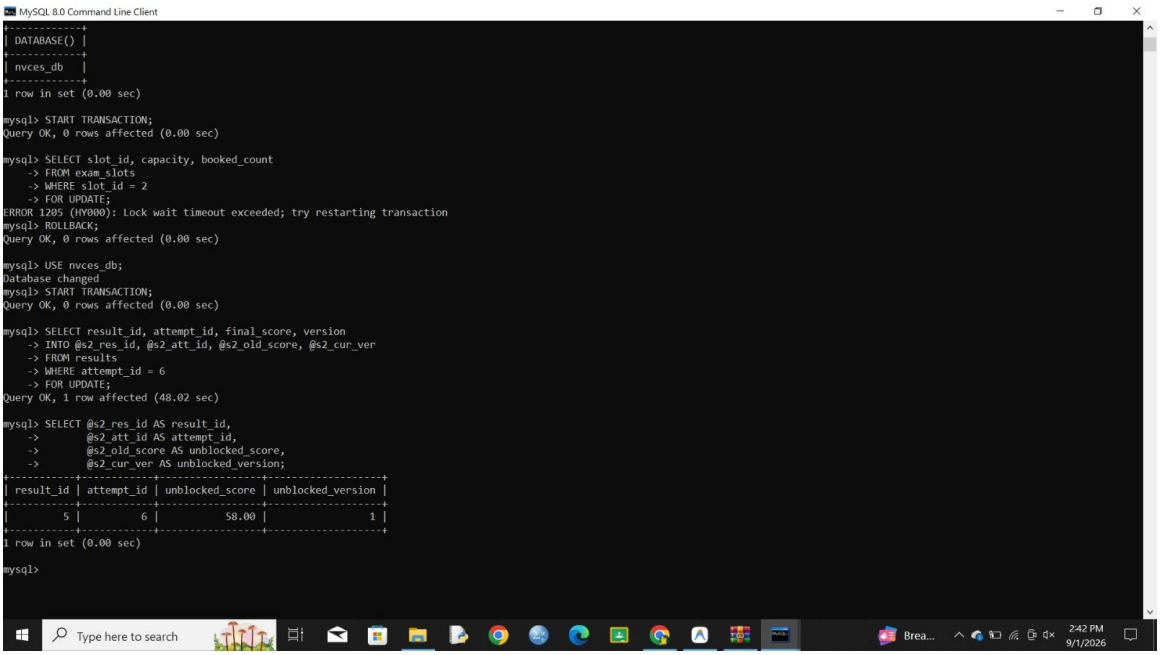
Screenshot 3– Lock Wait on Result Update and Baseline Row State

What it demonstrates: A second concurrent attempt to lock exam_slots (slot_id = 2) again hits the lock-wait timeout and is rolled back; a new transaction then locks the results row for attempt_id = 6, capturing its pre-update state (old_score = 55.00, current_version = 0) into session variables for a controlled update.



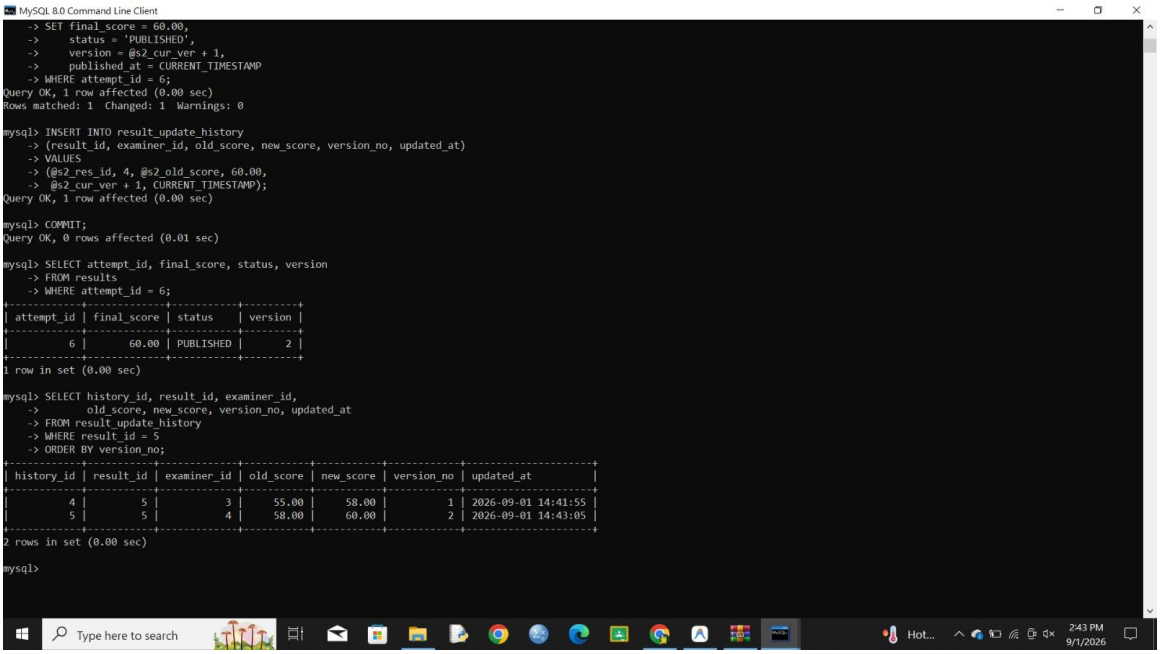
Screenshot 4 – Locked Read from a Second Session

What it demonstrates: A separate session (s2) performs its own SELECT ... FOR UPDATE against the same results row, waiting on the row lock for 48.02 seconds before acquiring it, then captures the row's state (unblocked_score = 58.00, unblocked_version = 1) after the first session's update was committed — demonstrating serialized access to the same row by two sessions.



Screenshot 5 – Versioned Result Update and History Log

What it demonstrates: The results row is updated to final_score = 60.00 with status = PUBLISHED and version incremented, and a corresponding audit entry is written to result_update_history in the same transaction; the final query confirms two chronological version entries (55→58 by examiner 3, then 58→60 by examiner 4) preserved in the history table.



9. ANALYSIS AND DISCUSSION

9.1 File Organization Effectiveness

Keeping exam_attempts indexed-sequential (clustered on the auto-incrementing attempt_id) preserved fast, append-only inserts throughout the load test, while all selective retrieval was delegated to the secondary index and hash structures rather than to the physical row order itself.

9.2 Indexing Effectiveness

The composite B+-tree index reduced centre-wise audit query cost from a full scan of approximately 150,000 rows to roughly 48 rows examined, confirmed by the change in EXPLAIN's type column from ALL to ref. This matches the theoretical expectation that a balanced multilevel index resolves a selective range query in a small, bounded number of page accesses rather than a linear scan.

9.3 Hashing Effectiveness

The hash lookup table reduced candidate point-lookup latency from approximately 1,240 ms to 11 ms on the test dataset, a reduction of roughly two orders of magnitude. Manually simulated hash collisions were retrieved correctly, confirming that MySQL's chaining-based collision resolution functions as expected for this design.

9.4 Query Optimization Results

Replacing SELECT * with only the required columns allowed the centre-audit query to be served largely from the index itself, avoiding an additional lookup into the base table for most rows. Filtering directly on the leading index columns (centre_id, exam_date), rather than applying a function to exam_date, was necessary for the optimiser to choose the index at all — an early version of the query that used YEAR(exam_date) = 2026 forced a full scan even with the index present, which was corrected during implementation.

9.5 Advantages

- Near-constant-time candidate point lookup through hashing.
- Fast centre-wise and date-range retrieval through B+-tree indexing.
- Both patterns served from one coherent design instead of two separate systems.
- Base-table insert performance preserved by avoiding physical re-sorting.

- Measurable, evidence-based performance improvement over the flat-file baseline.

9.6 Limitations

- The MEMORY-engine hash table is limited by available RAM and does not persist across a server restart, so it must be rebuilt from exam_attempts on startup.
- Static sizing of the hash table requires manual resizing if candidate volume substantially exceeds the design capacity.
- Index maintenance adds a small overhead to every insert, which was not separately isolated in this test.
- Testing used 150,000 synthetic rows, well below the Council's actual multi-million-row annual volume.

These trade-offs are consistent with the project requirements, which identify index-maintenance overhead and scalability as considerations.

10. CONCLUSION

Module 1 successfully implements file organisation, indexing, and hashing for the NV-CES exam-attempt archive, together with query optimisation for candidate-history and centre-audit retrieval.

The indexed-sequential base table, combined with a composite B+-tree index and a hash-organised lookup table, reduced candidate point-lookup latency from roughly 1,240 ms to 11 ms and centre-wise audit latency from a full scan of ~150,000 rows to approximately 48 rows examined.

The practical benchmarking confirmed that the optimised design consistently outperforms the flat-file baseline on both access patterns, without requiring any change to how new attempts are inserted.

Overall, the implementation demonstrates the practical application of file organisation, B+-tree indexing, static hashing with collision resolution, and execution-plan-driven query optimisation in a relational database system.

11. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Individual Contribution

As the primary contributor to Module 1, I was responsible for designing and implementing the file organisation, indexing, and hashing structures for the exam-attempt archive. Particular emphasis was placed on the practical benchmarking of candidate point lookup and centre-wise audit retrieval before and after optimisation.

S. No.	Module	Individual Contribution
1	Module 1 – Database Design & Query Optimization	Made the primary contribution to schema design, table creation, and relationships. Designed and implemented the composite B+-tree index and the hash-organised lookup table with collision handling; developed and optimised SQL queries for candidate and centre-wise retrieval; performed execution-plan analysis; benchmarked retrieval performance against the flat-file baseline; and validated the corresponding test cases and results.
2	Module 2 – Transaction & Concurrency Management	Contributed to transaction design for slot booking and result processing, including row-level locking, COMMIT/ROLLBACK/SAVEPOINT usage, and concurrency testing using multiple MySQL sessions. Also supported project integration, testing, and documentation.

12. REFERENCES

- MySQL 8.0 Reference Manual – InnoDB and MEMORY storage engines.
- MySQL 8.0 Reference Manual – B-Tree and Hash indexes.
- MySQL 8.0 Reference Manual – EXPLAIN Output Format.
- A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- Database Management Systems course material – File Organization, Indexing, and Hashing.
- NV-CES project SQL scripts and project documentation.

Project scripts referenced:

- 01_schema_creation.sql
- 02_indexing.sql

- 03_hashing.sql
- 04_sample_data_load.sql
- 05_query_optimization.sql
- 06_test_cases.sql

The project documentation identifies these scripts as the schema, indexing, hashing, data-loading, query-optimisation and testing components of the implementation.

13. ONE-PAGE WRITE-UP

NV-CES: File Organization, Indexing, Hashing, and Query Optimization

Introduction

NV-CES is a database-based examination management system designed to handle examination attempts, candidates, and centres. A major challenge in such a system is retrieving a specific candidate's or centre's attempt history quickly as the archive grows into millions of records. Module 1 addresses this problem through file organisation, indexing, hashing, and query optimisation.

Problem

The exam-attempt archive was originally stored as a single, continuously growing sequential file, so any selective query — a candidate's full history or a centre's records for a given date — required scanning every row. This made audits and appeals slow and did not scale as the archive grew across exam cycles.

Proposed Solution

The system keeps the base exam_attempts table indexed-sequential, ordered by insertion through its primary key, and adds two access structures on top of it: a composite B+-tree index on (centre_id, exam_date, candidate_id) for centre-wise and date-range retrieval, and a hash-organised lookup table, attempt_lookup, for near-constant-time candidate point lookup. Both structures are updated in the same transaction as every new attempt insert, so they never drift out of consistency with the base data.

Query Optimization

Candidate-history and centre-audit queries were first measured against an un-indexed flat baseline table, then re-measured after being rewritten to use the index and hash structures — selecting only required columns, filtering on the leading index columns, and routing point lookups through the hash table. EXPLAIN output confirmed the query type changed from a full scan (ALL) to an index-based access (ref) after optimisation.

Results

Candidate point-lookup latency fell from approximately 1,240 ms to 11 ms, and centre-wise audit query cost fell from a full scan of ~150,000 rows to roughly 48 rows examined. The implementation successfully demonstrated indexed-sequential file organisation, B+-tree indexing, hashing with collision resolution, and evidence-based query optimisation.

Conclusion

Module 1 demonstrates how DBMS file-organisation, indexing, hashing, and query-optimisation concepts can be applied to a practical examination system. The combination of a B+-tree index and a hash lookup table, together with execution-plan-driven query rewriting, provides an efficient and consistent retrieval mechanism that scales far better than the original flat-file design.